

Unit 8: Graphs (Week 7)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: Introduction to Graphs

Section 8.1

Trees model hierarchies. But what if relationships aren't hierarchical?

A tree has one root, one parent per node, no cycles.

What structure would you need for a road network, a social network, or the internet — where anything can connect to anything?

8.1 What Is a Graph?

A **graph** $G = (V, E)$ consists of:

- A set of **vertices** (nodes) V
- A set of **edges** E — pairs of vertices that are connected

Vertices: { A, B, C, D }

Edges: { (A,B), (A,C), (B,D), (C,D) }

```
A --- B
|     |
C --- D
```

Trees are a special case of graphs — connected, acyclic, with exactly $n-1$ edges for n vertices. Graphs have no such restrictions.

8.1 Graph Vocabulary

Term	Definition
Adjacent	Two vertices connected by an edge
Degree	Number of edges incident to a vertex
Path	Sequence of vertices connected by edges
Cycle	A path that begins and ends at the same vertex
Connected	Every vertex reachable from every other
Sparse	Few edges relative to vertices ($E \ll V^2$)
Dense	Many edges relative to vertices ($E \approx V^2$)

The number of possible edges in an undirected graph is $V(V-1)/2$. A **complete graph** has all of them.

8.1 Paths and Connectivity

A graph is **connected** if there is a path between every pair of vertices. Otherwise it has multiple **connected components**.

Connected:

A – B – C
|
D

Disconnected (2 components):

A – B D – E
| |
F

Many graph algorithms only make sense on connected graphs.

Simple path: no repeated vertices.

Simple cycle: no repeated vertices except start = end.

Part 2: Applications of Graphs

Section 8.2

 **What do GPS navigation, Facebook friends, and package dependencies all have in common?**

They all involve relationships between things, and those relationships aren't always trees.

8.2 Real-world Entities

Graphs model any domain where **relationships between entities** matter.

Domain	Vertices	Edges
Road network	Intersections	Roads
Social network	People	Friendships
Internet	Routers	Links
Dependency manager	Packages	"Requires"
Compiler	Variables	Data flow
Scheduling	Tasks	Constraints

8.2 Graph Problems You Already Care About

- **Shortest path** — GPS routing, network latency
- **Reachability** — can user A reach resource B?
- **Cycle detection** — is there a circular dependency in this package?
- **Topological sort** — in what order should tasks be completed?
- **Minimum spanning tree** — cheapest way to connect all nodes (network cabling)
- **Connected components** — how many isolated clusters exist?

Graph algorithms are the engine behind Google Maps, Git's dependency resolution, LinkedIn's "people you may know," and CPU instruction scheduling.

Part 3: Adjacency Lists

Section 8.3



You need to store which vertices are connected to each other.

For a graph with 1,000 vertices and 2,000 edges, a vertex may connect to only 2 or 3 others on average.

Would you store the full grid of all possible vertex pairs? What's the cost?

8.3 Adjacency List Representation

Each vertex stores a **list of its neighbors**.

Graph: A – B – D
 | |
 C – – +

Adjacency list:

A → [B, C]
B → [A, C, D]
C → [A, B]
D → [B]

Can use linked lists or vectors.

8.3 Adjacency List: Cost Analysis

Operation	Cost
Store the graph	$O(V + E)$
Check if edge (u, v) exists	$O(\text{degree}(u))$
Iterate all neighbors of u	$O(\text{degree}(u))$
Iterate all edges	$O(V + E)$

Space is **$O(V + E)$** — proportional to what actually exists, not all possible edges.

Adjacency lists are the default choice for **sparse graphs**. Most real-world graphs (road networks, social networks, the web) are sparse — each vertex connects to a tiny fraction of all others.

8.3 Adding Edges

```
vector<vector<int>> adj(V);  
  
void addEdge(int u, int v) {  
    adj[u].push_back(v);  
    adj[v].push_back(u); // omit for directed graphs  
}
```

For an undirected graph, every edge (u, v) is stored **twice** — once in `adj[u]` and once in `adj[v]`. Space is still $O(V + E)$.

Part 4: Adjacency Matrices

Section 8.4



Checking whether edge (u, v) exists in an adjacency list takes $O(\text{degree}(u))$.

For a dense graph with millions of edge-existence queries,
that could be slow.

**What data structure gives $O(1)$ edge lookup — and
what does it cost?**

8.4 Adjacency Matrix Representation

A $V \times V$ matrix where `matrix[u][v] = 1` if edge (u, v) exists, 0 otherwise.

	A	B	C	D
A	[0	1	1	0]
B	[1	0	1	1]
C	[1	1	0	0]
D	[0	1	0	0]

Edge lookup: `matrix[u][v]` — **$O(1)$** , a single array access.

Space: **$O(V^2)$** — all possible pairs, whether edges exist or not.

8.4 Adjacency Matrix: Cost Analysis

Operation	Cost
Store the graph	$O(V^2)$
Check if edge (u, v) exists	$O(1)$
Iterate all neighbors of u	$O(V)$
Iterate all edges	$O(V^2)$

Adjacency matrices are preferred for **dense graphs** ($E \approx V^2$) or when $O(1)$ edge lookup is critical. For sparse graphs, iterating all neighbors costs $O(V)$ even if a vertex has only 2 neighbors, which is wasteful.

8.4 Choosing a Representation

Property	Adjacency List	Adjacency Matrix
Space	$O(V + E)$	$O(V^2)$
Edge lookup	$O(\text{degree})$	$O(1)$
Neighbor iteration	$O(\text{degree})$	$O(V)$
Best for	Sparse graphs	Dense graphs
Example	Road network	Flight connections

For most problems you encounter, **adjacency lists are the right default**. Switch to a matrix only when you need constant-time edge existence checks.

Part 6: Directed Graphs

Section 8.6



A friendship on Facebook is mutual. A follow on Twitter is not.

In a road network, some streets are one-way.

How does the graph model change when edges have direction?

8.6 Directed Graphs (Digraphs)

In a **directed graph**, each edge ($u \rightarrow v$) goes one way only. u can reach v , but v cannot necessarily reach u .

```
A → B → D
↓       ↑
C -----+
```

Adjacency list:

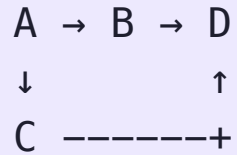
```
A → [B, C]
B → [D]
C → [D]
D → []
```

Implementation change: `addEdge(u, v)` inserts into `adj[u]` only — **no reverse insertion**.

8.6 In-Degree and Out-Degree

Each vertex in a digraph has two degree measures:

- **Out-degree:** number of edges leaving the vertex
- **In-degree:** number of edges entering the vertex



Vertex	Out	In
A	2	0
B	1	1
C	1	1
D	0	2

8.6 Directed Acyclic Graphs (DAGs)

A **DAG** is a directed graph with no cycles. DAGs model any process where order matters and circular dependencies are forbidden.

- Build systems (Makefile targets)
- Course prerequisites
- Git commit history
- Spreadsheet cell formulas

```
CPTR 131 → CPTR 231 → CPTR 242
           ↓
        CPTR 332
```

Topological sort produces a linear ordering of vertices such that every edge $u \rightarrow v$ places u before v . Only possible on DAGs.

8.6 Cycle Detection

A directed graph has a cycle if DFS ever encounters a vertex already on the **current recursion stack** (a "back edge").

```
Visit A → visit B → visit D → visit B again
                                ↑ already on stack → CYCLE
```

```
bool hasCycle(int u, vector<bool>& visited, vector<bool>& onStack) {
    visited[u] = onStack[u] = true;
    for (auto& e : adj[u])
        if (!visited[e.to] && hasCycle(e.to, visited, onStack)) return true;
        else if (onStack[e.to]) return true;
    return onStack[u] = false;
}
```

Part 7: Weighted Graphs

Section 8.7

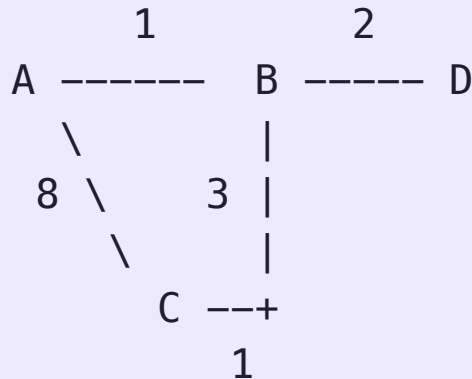
 **In a road network, not all roads are the same length.**

BFS finds the path with the fewest hops but not necessarily the shortest distance.

What information does the graph need to support shortest-distance queries?

8.7 Weighted Graphs

A **weighted graph** assigns a numeric **weight** (cost, distance, capacity) to each edge.



BFS is blind to weights. **Dijkstra's algorithm** uses a priority queue to always expand the lowest-cost frontier.

8.7 Weighted Adjacency List

```
struct Edge { int to, weight; };  
vector<vector<Edge>> adj(V);  
  
void addEdge(int u, int v, int w) {  
    adj[u].push_back({v, w});  
    adj[v].push_back({u, w});  
}
```

The only change from an unweighted list: edges carry a `weight` field. Algorithms that don't use weights simply ignore it, the same representation serves both.

8.7 Weighted Adjacency Matrix

For weighted graphs, the matrix stores **edge weights** instead of 0/1 flags.

	A	B	C	D
A	[0	4	7	∞]
B	[4	0	3	2]
C	[7	3	0	∞]
D	[∞	2	∞	0]

∞ (or a sentinel like `INT_MAX`) means no edge. `matrix[u][v]` gives the direct cost from `u` to `v` in $O(1)$, which is convenient for dense weighted graphs and Floyd-Warshall all-pairs shortest path.

8.7 Negative Weights

Dijkstra's algorithm assumes **non-negative weights**. Negative weights break its greedy assumption.

Algorithm	Weights	Complexity
BFS	Unweighted	$O(V + E)$
Dijkstra	Non-negative	$O((V + E) \log V)$
Bellman-Ford	Any (detects neg. cycles)	$O(VE)$
Floyd-Warshall	Any (all pairs)	$O(V^3)$

Negative weights appear in real problems: profit/loss graphs, currency arbitrage, game state scoring. Bellman-Ford handles them at higher cost.

Unit 8 Summary

Concept	Key Idea
Graph $G = (V, E)$	Vertices + edges; trees are a special case
Adjacency list	$O(V + E)$ space; default for sparse graphs
Adjacency matrix	$O(V^2)$ space; $O(1)$ edge lookup; dense graphs
Directed graph	Edges have direction; in/out degree; DAGs
Weighted graph	Edges carry cost; BFS insufficient for shortest path
Cycle	Path from a vertex back to itself
DAG	Directed, acyclic; enables topological sort

Next: Graph Traversal Algorithms
